

Práctica 4: Ramificación y Poda, Programación Lineal

Algoritmia Básica

Imad Habib Jali (901421)

Jorge Bellido Lobera (903080)

1. Diseño del Algoritmo de Ramificación y Poda

El problema consiste en particionar un conjunto de N participantes en $N/3$ equipos de 3 personas, minimizando el conflicto total. Este problema es equivalente a encontrar una partición de un grafo completo ponderado en clicas de tamaño 3 con peso mínimo.

1.1. Representación de Soluciones y Árbol de Búsqueda

La solución se construye asignando participantes $0, 1, \dots, N - 1$ de forma secuencial. Un estado del árbol de búsqueda se define por:

- El participante p que se está asignando actualmente (nivel del árbol).
- La configuración actual de los equipos parciales.
- El conflicto acumulado hasta el momento.

La dimensión del espacio de búsqueda (número de formas de particionar N personas en equipos de 3) viene dada por:

$$|S| = \frac{N!}{(N/3)! \cdot (3!)^{N/3}}$$

Para los valores de prueba habituales, esta dimensión crece de forma explosiva:

- $N = 6 \implies |S| = 10$
- $N = 9 \implies |S| = 280$
- $N = 12 \implies |S| = 15,400$
- $N = 15 \implies |S| = 1,401,400$
- $N = 21 \implies |S| \approx 1,05 \times 10^{10}$

Para reducir este espacio, se imponen las siguientes reglas:

1. Un participante solo puede unirse a un equipo ya existente que no esté lleno o iniciar el primer equipo vacío disponible (rompiendo la simetría de los IDs de los equipos).
2. Los equipos se completan siguiendo el orden de los participantes.

1.2. Función de Cota Inferior (Lower Bound)

Para podar ramas no prometedoras, se utiliza una cota inferior $h(n)$ que estima el conflicto mínimo que añadirán los participantes restantes. Sea $W_{i,j} = C_{i,j} + C_{j,i}$ el conflicto mutuo entre i y j . El conflicto total G se puede expresar como $\frac{1}{2} \sum_{i=0}^{N-1}$ conflicto de i con sus dos compañeros.

La cota inferior se calcula sumando para cada participante i el conflicto mínimo con sus socios necesarios:

- Si i ya tiene 2 socios en su equipo, no añade nada a la cota futura.
- Si i tiene 1 socio j , añade al menos $W_{i,j} + \min_k W_{i,k}$.
- Si i no tiene socios asignados, añade al menos la suma de sus dos menores $W_{i,k}$.

La suma de estas estimaciones se divide por 2 para obtener el nivel de conflicto (ya que cada par $\{i, j\}$ se contaría dos veces).

2. Implementación

El algoritmo se ha implementado en C++ utilizando un enfoque de **Backtracking con Poda (DFS Branch & Bound)**. Se ha optimizado la actualización de la cota inferior de forma incremental en cada paso de la recursión, evitando el recálculo total.

Además, se realiza una fase inicial de **Greedy Search** para obtener una cota superior (Upper Bound) de calidad desde el inicio, lo que permite podar ramas ineficientes de manera muy temprana.

3. Experimentación

Se han realizado pruebas con diferentes tamaños de N para evaluar el rendimiento y comparar el algoritmo de Ramificación y Poda (B&B) con el modelo de Programación Lineal (PLI).

N	B&B Tiempo (ms)	B&B Nodos	PLI Tiempo (ms)	Valor Óptimo
6	0	22	84*	42
9	0	232	145	73
12	0	868	70	70
15	2	10,630	168	68
18	64	285,725	409	74
21	2,997	12,811,209	1,118	112
24	14,492	47,862,255	2,104	96

* Nota: El tiempo de 84 ms para $N=6$ en PLI incluye el overhead de inicialización de la librería PuLP y el solver en Python.

Se observa que para valores pequeños de N ($N \leq 18$), el algoritmo B&B es superior debido a la eficiencia de la cota inferior y la baja sobrecarga del lenguaje C++. Sin embargo, a medida que N aumenta, el crecimiento exponencial del árbol de búsqueda hace que el tiempo de ejecución de B&B aumente drásticamente. Se identifica un punto de cruce algorítmico entre $N = 18$ y $N = 21$. En $N = 21$, el B&B genera más de 12 millones de nodos, lo que eleva su tiempo a casi 3 segundos, siendo ya superado por el solver de PLI (1.1 segundos). En $N = 24$, la diferencia es de un orden de magnitud (14.5s vs 2.1s).

4. Programación Lineal Entera (PLI)

Se ha desarrollado una solución alternativa utilizando el modelo de programación lineal entera. La formulación utiliza variables binarias $z_{i,j}$ que indican si los participantes i y j pertenecen al mismo equipo.

Modelo:

- **Minimizar:** $\sum_{i < j} (C_{i,j} + C_{j,i}) \cdot z_{i,j}$
- **Sujeto a:**
 - $\sum_{j \neq i} z_{i,j} = 2, \quad \forall i$ (Cada uno tiene exactamente 2 socios).
 - $z_{i,j} + z_{j,k} - 1 \leq z_{i,k}, \quad \forall i, j, k$ (Transitividad/Clicas de 3).

Este modelo se implementó en Python con la librería PuLP, validando los resultados obtenidos por el algoritmo de Ramificación y Poda.

5. Conclusiones

El algoritmo de Ramificación y Poda es altamente dependiente de la calidad de la cota inferior. Con una cota basada en los pesos mínimos del grafo, el espacio de búsqueda se reduce drásticamente. La programación lineal ofrece una forma robusta de validar la correctitud del algoritmo principal, permitiendo asegurar que las podas realizadas no eliminan soluciones óptimas.